

Oracle Bulk Collection

BULK COLLECT & FORALL — high-performance PL/SQL · Oracle 19c

Row-by-row PL/SQL loops cause many **context switches** between the PL/SQL engine and the SQL engine. **BULK COLLECT** (bulk fetch) and **FORALL** (bulk DML) move data in batches, cutting context switches dramatically — often 10×–50× faster for large volumes.

1. The Problem: Context Switching

```
-- SLOW: one context switch per row
FOR r IN (SELECT id, sal FROM emp) LOOP
    UPDATE emp_stg SET sal = r.sal * 1.1 WHERE id = r.id;  -- switch each iteration
END LOOP;
```

2. BULK COLLECT — Bulk Fetch

```
DECLARE
    TYPE t_emp IS TABLE OF emp%ROWTYPE;
    l_emp t_emp;
BEGIN
    SELECT * BULK COLLECT INTO l_emp FROM emp;      -- all rows in one fetch
    DBMS_OUTPUT.put_line('Fetched: ' || l_emp.COUNT);
END;
/
```

Also works with a cursor and **FETCH ... BULK COLLECT INTO**, and with multiple collections:

```
SELECT id, name BULK COLLECT INTO l_ids, l_names FROM customer;
```

Memory warning: **BULK COLLECT** without a **LIMIT** loads *all* rows into PGA memory. For large tables, always fetch in batches with the **LIMIT** clause (see the "Limited" guide).

3. FORALL — Bulk DML

```
DECLARE
    TYPE t_id IS TABLE OF emp.id%TYPE;
    TYPE t_sal IS TABLE OF emp.sal%TYPE;
    l_id t_id; l_sal t_sal;
BEGIN
    SELECT id, sal * 1.1 BULK COLLECT INTO l_id, l_sal FROM emp;

    FORALL i IN 1 .. l_id.COUNT      -- one bulk DML, not a loop
        UPDATE emp_stg SET sal = l_sal(i) WHERE id = l_id(i);

    DBMS_OUTPUT.put_line('Rows updated: ' || SQL%ROWCOUNT);
END;
/
```

FORALL with INSERT from a collection of records

```
FORALL i IN 1 .. l_emp.COUNT
    INSERT INTO emp_archive VALUES l_emp(i);      -- record insert
```

4. FORALL for Sparse Collections

If the driving collection has gaps (after deletes), use **INDICES OF** or **VALUES OF**:

```

FORALL i IN INDICES OF l_id                                -- skips deleted elements
  UPDATE emp_stg SET sal = l_sal(i) WHERE id = l_id(i);

FORALL i IN VALUES OF l_index_coll                        -- use indexes listed in another collection
  INSERT INTO tgt VALUES l_data(i);

```

5. Useful Bulk Attributes

Attribute	Meaning
<code>SQL%ROWCOUNT</code>	Total rows affected by the whole FORALL
<code>SQL%BULK_ROWCOUNT(i)</code>	Rows affected by the i-th FORALL statement
<code>SQL%BULK_EXCEPTIONS</code>	Errors collected when using SAVE EXCEPTIONS (see "Rejected/Limited" guide)

```

FORALL i IN 1 .. l_id.COUNT
  UPDATE emp_stg SET sal = l_sal(i) WHERE id = l_id(i);

FOR i IN 1 .. l_id.COUNT LOOP
  DBMS_OUTPUT.put_line('Stmt '||i||' updated '||SQL%BULK_ROWCOUNT(i));
END LOOP;

```

6. RETURNING with BULK COLLECT

```

FORALL i IN 1 .. l_id.COUNT
  DELETE FROM emp_stg WHERE id = l_id(i)
  RETURNING id BULK COLLECT INTO l_deleted;           -- capture affected rows

```

Best practices

- Use `LIMIT` (e.g. 100–1000) with a loop for large data sets to cap PGA memory.
- `FORALL` runs *one* DML statement per collection — no loop body, only a single DML.
- Combine with `SAVE EXCEPTIONS` so one bad row doesn't abort the whole batch.
- Prefer plain SQL (`INSERT ... SELECT`) when no procedural logic is needed — it's still fastest.